

Programming Assignment #2

CIS 436 – Mobile Application Development
University of Michigan - Dearborn
Prof. John P. Baugh

Points: _____ / 100

Objectives

- To create a basic user interface
- To create event listeners and handlers for UI components
- To create a basic test plan
- To work with fragments
- To handle orientation changes

Instructions

You will develop a **Tic-Tac-Toe** mobile game where the user plays against another person (local multiplayer) or against a simple AI opponent.

The app should:

- Allow users to **tap a grid cell** to make a move.
- Show **X or O** based on the player's turn.
- Determine and display a **winner or a tie**.
- Handle **restarts and resets**.
- Maintain the game state during **orientation changes**.

Functional Requirements

1. User Interface

- The UI must include:
 - A **3x3 grid** of buttons or ImageViews for Tic-Tac-Toe.
 - A **text view** or similar element to display the current player's turn.
 - A **reset button** to restart the game.
 - A **score tracker** (optional bonus).
 - A **mode selector** (optional: Player vs. Player or Player vs. AI).
- The app **must not** use the keyboard for input; interactions must be done via **tapping the grid cells**.

2. Game Mechanics

- The game must enforce **Tic-Tac-Toe rules**:
 - **Players** (user and the AI) **take turns** marking the grid with X or O.
 - **The first player to align three marks in a row, column, or diagonal wins.**
 - If all grid spaces are filled and no player wins, the game ends in a **draw**.
- If implementing an **AI opponent**:
 - The “AI” can use **random moves** (placing the symbol into a randomly chosen open slot)

3. Fragment Usage

- The app **must use fragments** to separate concerns:
 - **Top Fragment:** Displays game status (current turn, winner, or tie).
 - **Bottom Fragment:** Contains the 3x3 grid of buttons.

4. Orientation Handling

- When rotating the device, the app **must retain**:
 - The current game board state.
 - The current player’s turn.
 - The game result if the match is over.

5. Test Plan

- Students must provide a **test plan** in an Excel workbook (or PDF of the test cases)
- The test plan should have **at least three days** of testing logs.
- Each test entry should contain:
 - The test case (e.g., "Player X taps top-left cell").
 - The expected outcome (e.g., "X appears in the top-left cell").
 - The actual outcome (e.g., "X appears correctly").
 - Whether the test passed or failed.

Sample UI

The following shows a simple series of moves on the game board. At any given time, the game board with its nine spaces will be viewable.



- You must use: **buttons, text views** to complete the assignment
- You must provide a test plan with at least **three days** on separate worksheet tabs indicating the tests that were performed
- You **must use fragments** (for the buttons in the actual game, as well as restarting, etc.)
 - If you do not use fragments, you will lose a significant number of points

Deliverables

Zip your entire Android Studio project, with the **test plan stored at the top level of the project**.

Upload the entire zip file to Canvas on or before the due date.

Grading Criteria/Rubric

Criteria	Points
User Interface & Design	20
Event Handling	20
Fragment Implementation	20
Orientation Handling	15
Test Plan Submission	15
Code Quality & Comments	10
Total	100